

Hydrozoan Artifact: Testbed Details

This note accompanies the paper *Hydrozoan: Latency-Adaptive DAG Consensus under Mixed Byzantine and Crash Faults*. Sections, lemmas, theorems, algorithms, tables and figures numbered without the prefix “T” refer to the paper.

T. Testbed Details

We detail the testbed and the experimental setup used to evaluate Hydrozoan.

We deploy all protocols on AWS, using `m5d.8xlarge` instances across six regions: Northern Virginia (us-east-1), Ohio (us-east-2), Frankfurt (eu-central-1), London (eu-west-2), Paris (eu-west-3) and Tokyo (ap-northeast-1). Each machine provides 10 Gbps of bandwidth, 32 virtual CPUs (16 physical cores) on a 3.1 GHz Intel Xeon Skylake 8175M, 128 GB of memory, and runs Ubuntu server 24.04. Validators are assigned to regions round-robin in the order above, so at $n = 50$ the regions hold 9, 9, 8, 8, 8 and 8 validators: Tokyo is a tail of 8 against 42 nearby validators, and 1 of 10 in the small committees. We measured the round-trip times between all instances on the day of the runs: the nearby validators are within 96 ms of each other (US–EU 75–96 ms, intra-Europe 7–12 ms, Virginia–Ohio 11 ms), Tokyo is 135–238 ms from every other region, and intra-region round trips are under 1 ms. This matrix is what Section S calibrates the simulator with.

A load generator collocated with each validator submits 512-byte transactions in an open loop at a fixed rate. Every protocol runs with two leaders per round and a leader timeout of 1 s. Runs last 240–300 s at $n = 50$ with a 120 s load ramp, and 120 s at $n \leq 10$ with a 30 s ramp; the length is chosen so that every run shows a steady state after the ramp, since the throughput plateau is flat and a longer run only adds cost. Prometheus scrapes every validator every 15 s. Crash injection kills nearby validators first, in region order; the tail always survives. Crashed validators’ load generators die with them, so the throughput at x crashes is $(50 - x)/50$ of the offered load by construction.

End-to-end latency is measured from submission to commit, leader block latency from the signed block timestamp to the commit of the block; their difference is queuing before inclusion in a block, 95–150 ms in every fault-free run of Section 8. For per-run statistics we discard the scrapes before the ramp, restrict each run to its throughput plateau (scrapes at $\geq 90\%$ of the maximum) and report the floor of the per-scrape across-validator percentiles (p50 and p90) and the median sustained throughput. The share of leader slots decided by each rule is read from the committer’s counters over the same window.

Hydrangea [1] is measured from its own codebase: an $n = 10$ curve (~ 330 ms p50 at 10–70k tx/s, 2.6 s at 100k), a table row for $n = 50$ (~ 67 s at every load) and one for 2 crashes at $n = 10$ (27–76 s). It supports the throughput-ceiling claim of Section 2 and is never plotted at a point it did not measure.

References

- [1] N. Shrestha, A. Kate, and K. Nayak, “Hydrangea: Optimistic two-round partial synchrony with improved fault resilience,” in *USENIX Security*, 2026, pp. 3677–3694.